Final Project

Scraping data from various websites to retrieve player contract information, team schedules, and the state that teams reside in

4/28/2026

Group Members:

- Sam Papaleo
- Josiah Nelson
- Erik Modesto

```
# importing necessary packages
from urllib.request import urlopen
import time
from bs4 import BeautifulSoup, NavigableString, Tag
import pandas as pd
import re
```

Scraping Contracts

In this section we are scraping data from overthecap.com, and extracting the table that contains all current NFL players and their respective contract information. We then add this information into a pandas dataframe to allow for future data analysis/manipulation.

```
# scraping website to get player contract info
url = "https://overthecap.com/contracts/buffalo-bills"
html = urlopen(url)

bs = BeautifulSoup(html, "html.parser")

# Find the table with the specified class
table = bs.find("table", class_="sortable controls-table")

# Extract table headers
headers = [th.get_text(strip=True) for th in table.find("thead").find_all("th")]

# Extract table rows
rows = []
for tr in table.find("tbody").find_all("tr"):
    row_data = [td.get_text(strip=True) for td in tr.find_all("td")]
    rows.append(row_data)

# Create a Pandas DataFrame
df = pd.DataFrame(rows, columns=headers)
display(df.head())
```

	Player	Pos.	Team	Total Value	APY	TotalGuaranteed	Avg.Guarantee/Year	% Guaranteed
0	Dak Prescott	QB	Cowboys	$240,000,000	$60,000,000	$129,000,000	$32,250,000	53.8%
1	Trevor Lawrence	QB	Jaguars	$275,000,000	$55,000,000	$142,000,000	$28,400,000	51.6%
2	Joe Burrow	QB	Bengals	$275,000,000	$55,000,000	$146,510,000	$29,302,000	53.3%
3	Jordan Love	QB	Packers	$220,000,000	$55,000,000	$100,800,000	$25,200,000	45.8%
4	Josh Allen	QB	Bills	$330,000,000	$55,000,000	$147,000,000	$24,500,000	44.5%

```
#Standardizing the value columns
columns_standardizing = ['Total Value', 'APY', 'TotalGuaranteed', 'Avg.Guarantee/Year', '% Guaranteed']
```

```
for col in columns_standardizing:
        df[col] = df[col].astype(str).str.replace(r'[$,%]', '', regex=True).astype(float)

# Selecting top 1000 players by APY (keeping large number of observations without having to deal with nonsensical contr
df_top_1000 = df.sort_values(by='APY', ascending=False).head(1000)

display(df_top_1000.head())
```

	Player	Pos.	Team	Total Value	APY	TotalGuaranteed	Avg.Guarantee/Year	% Guaranteed
0	Dak Prescott	QB	Cowboys	240000000.0	60000000.0	129000000.0	32250000.0	53.8
1	Trevor Lawrence	QB	Jaguars	275000000.0	55000000.0	142000000.0	28400000.0	51.6
2	Joe Burrow	QB	Bengals	275000000.0	55000000.0	146510000.0	29302000.0	53.3
4	Josh Allen	QB	Bills	330000000.0	55000000.0	147000000.0	24500000.0	44.5
3	Jordan Love	QB	Packers	220000000.0	55000000.0	100800000.0	25200000.0	45.8

Get NFL Weekly Schedule For Each Team

In this section we are scraping the 2026 schedule from the NFL website, and creating a table to hold said schedule. The home team is represented by each column, with the 32 columns representing the 32 NFL teams. Each row represents that home team's matchup (9 rows for 9 home games).

```
import pandas as pd

# Since ESPN blocks scraping, the 2026 home/away opponent data is hardcoded below
# Source: https://www.espn.com/nfl/story/_/id/47492947/2026-nfl-schedule-regular-season-opponents-all-32-teams

schedule = {
    "Cowboys":    ["Giants", "Eagles", "Commanders", "Texans", "Colts", "Rams", "Seahawks", "Packers", ""],
    "Giants":     ["Cowboys", "Eagles", "Commanders", "Texans", "Colts", "Rams", "Seahawks", "Lions", ""],
    "Eagles":     ["Cowboys", "Giants", "Commanders", "Cardinals", "Jaguars", "49ers", "Titans", "Buccaneers", "Ravens
    "Commanders": ["Cowboys", "Giants", "Eagles", "Cardinals", "Jaguars", "49ers", "Titans", "Browns", "Saints"],
    "Bears":      ["Lions", "Packers", "Vikings", "Bills", "Dolphins", "Falcons", "Panthers", "Seahawks", ""],
    "Lions":      ["Bears", "Packers", "Vikings", "Patriots", "Saints", "Jets", "Buccaneers", "Rams", ""],
    "Packers":    ["Bears", "Lions", "Vikings", "Falcons", "Bills", "Panthers", "Dolphins", "Cardinals", ""],
    "Vikings":    ["Bears", "Lions", "Packers", "Patriots", "Saints", "Jets", "Buccaneers", "49ers", ""],
    "Falcons":    ["Panthers", "Saints", "Buccaneers", "Browns", "Packers", "Vikings", "Steelers", "Commanders", ""],
    "Panthers":   ["Falcons", "Saints", "Buccaneers", "Browns", "Packers", "Vikings", "Steelers", "Eagles", ""],
    "Saints":     ["Falcons", "Panthers", "Buccaneers", "Ravens", "Bears", "Bengals", "Lions", "Giants", ""],
    "Buccaneers": ["Falcons", "Panthers", "Saints", "Ravens", "Bears", "Bengals", "Lions", "Cowboys", ""],
    "Cardinals":  ["Rams", "49ers", "Seahawks", "Broncos", "Raiders", "Eagles", "Commanders", "Buccaneers", ""],
    "Rams":       ["Cardinals", "49ers", "Seahawks", "Cowboys", "Chiefs", "Chargers", "Giants", "Saints", ""],
    "49ers":      ["Cardinals", "Rams", "Seahawks", "Cowboys", "Chiefs", "Chargers", "Giants", "Falcons", ""],
    "Seahawks":   ["Cardinals", "Rams", "49ers", "Broncos", "Raiders", "Eagles", "Commanders", "Panthers", ""],
    "Ravens":     ["Browns", "Steelers", "Bengals", "Jaguars", "Titans", "Dolphins", "Falcons", "Panthers", "Commander
    "Browns":     ["Ravens", "Steelers", "Bengals", "Texans", "Colts", "Patriots", "Commanders", "Eagles", ""],
    "Steelers":   ["Ravens", "Browns", "Bengals", "Jaguars", "Titans", "Jets", "Saints", "Buccaneers", "Giants"],
    "Bengals":    ["Ravens", "Browns", "Steelers", "Jaguars", "Titans", "Patriots", "Saints", "Buccaneers", "Eagles"],
    "Texans":     ["Colts", "Jaguars", "Titans", "Browns", "Steelers", "Chiefs", "Commanders", "Eagles", "Vikings"],
    "Colts":      ["Texans", "Jaguars", "Titans", "Browns", "Steelers", "Chargers", "Commanders", "Eagles", "Packers"]
    "Jaguars":    ["Texans", "Colts", "Titans", "Ravens", "Bengals", "Broncos", "Cowboys", "Giants", ""],
    "Titans":     ["Texans", "Colts", "Jaguars", "Ravens", "Bengals", "Patriots", "Cowboys", "Giants", ""],
    "Chiefs":     ["Raiders", "Chargers", "Broncos", "Cardinals", "Panthers", "Patriots", "Jets", "Steelers", "49ers"]
    "Raiders":    ["Chiefs", "Chargers", "Broncos", "Ravens", "Bills", "Rams", "Dolphins", "Seahawks", "Buccaneers"],
    "Chargers":   ["Chiefs", "Raiders", "Broncos", "Cardinals", "Colts", "Patriots", "Jets", "49ers", ""],
    "Broncos":    ["Chiefs", "Raiders", "Chargers", "Ravens", "Bills", "Rams", "Dolphins", "Seahawks", ""],
    "Bills":      ["Dolphins", "Patriots", "Jets", "Bears", "Lions", "Jaguars", "Chiefs", "Chargers", "Seahawks"],
    "Dolphins":   ["Bills", "Patriots", "Jets", "Broncos", "Packers", "Texans", "Raiders", "Rams", "Vikings"],
    "Patriots":   ["Bills", "Dolphins", "Jets", "Bears", "Bengals", "Lions", "Chiefs", "Chargers", ""],
    "Jets":       ["Bills", "Dolphins", "Patriots", "Cardinals", "Bears", "Lions", "Chiefs", "Chargers", "Titans"],
}

df = pd.DataFrame(schedule)
```

```
df_opponents = df.copy()
display(df)
```

	Cowboys	Giants	Eagles	Commanders	Bears	Lions	Packers	Vikings	Falcons	Panthers
0	Giants	Cowboys	Cowboys	Cowboys	Lions	Bears	Bears	Bears	Panthers	Falcons
1	Eagles	Eagles	Giants	Giants	Packers	Packers	Lions	Lions	Saints	Saints
2	Commanders	Commanders	Commanders	Eagles	Vikings	Vikings	Vikings	Packers	Buccaneers	Buccaneers
3	Texans	Texans	Cardinals	Cardinals	Bills	Patriots	Falcons	Patriots	Browns	Browns
4	Colts	Colts	Jaguars	Jaguars	Dolphins	Saints	Bills	Saints	Packers	Packers
5	Rams	Rams	49ers	49ers	Falcons	Jets	Panthers	Jets	Vikings	Vikings
6	Seahawks	Seahawks	Titans	Titans	Panthers	Buccaneers	Dolphins	Buccaneers	Steelers	Steelers
7	Packers	Lions	Buccaneers	Browns	Seahawks	Rams	Cardinals	49ers	Commanders	Eagles
8			Ravens	Saints						

9 rows × 32 columns

Getting Team-By-State

In this section we are extracting the table from the website into a pandas dataframe, as we plan on merging these dataframes into one large dataframe ready for data analysis/manipulation. We chose the table whose class was "content3" to make sure we were extracting the correct table, and then went through each row to extract the state name for each respective team, and standardize the names to just be represented by their mascot.

```
#scraping nfl teams by state website and extracting table
url_states = "https://state.1keydata.com/nfl-teams-by-state.php"
states = urlopen(url_states)
bs_states = BeautifulSoup(states, "html.parser")

table_states = bs_states.find("table", class_="content3")
#extracting table into a pandas dataframe
if table_states:
    states_teams = [] #skipping header row
    for tr in table_states.find_all("tr")[1:]: #going through each table row
        tds = tr.find_all("td")
        if len(tds) == 2:
            state_name = tds[0].get_text(strip=True)
            team_elements = [t.get_text(strip=True) for t in tds[1].find_all(text=True) if t.strip()]
            for team_name in team_elements:
                states_teams.append({"Team": team_name, "State": state_name})
    df_states = pd.DataFrame(states_teams)
else:
    print("Error Getting Info")
# Changing 'Washington Football Team' to 'Commanders'
df_states['Team'] = df_states['Team'].replace('Washington Football Team', 'Commanders')

# Extracting mascot name for teams
df_states['Team'] = df_states['Team'].apply(lambda x: x.split()[-1])

display(df_states)
```

```
/tmp/ipykernel_6085/2619777037.py:14: DeprecationWarning: The 'text' argument to find()-type methods is deprecated. Use
  team_elements = [t.get_text(strip=True) for t in tds[1].find_all(text=True) if t.strip()]
```

	Team	State
0	Cardinals	Arizona
1	Rams	California
2	Chargers	California
3	49ers	California
4	Broncos	Colorado
5	Jaguars	Florida
6	Dolphins	Florida
7	Buccaneers	Florida
8	Falcons	Georgia
9	Bears	Illinois
10	Colts	Indiana
11	Saints	Louisiana
12	Ravens	Maryland
13	Commanders	Maryland
14	Patriots	Massachusetts
15	Lions	Michigan
16	Vikings	Minnesota
17	Chiefs	Missouri
18	Raiders	Nevada
19	Giants	New Jersey
20	Jets	New Jersey
21	Bills	New York
22	Panthers	North Carolina
23	Bengals	Ohio
24	Browns	Ohio
25	Eagles	Pennsylvania
26	Steelers	Pennsylvania
27	Titans	Tennessee
28	Cowboys	Texas
29	Texans	Texas
30	Seahawks	Washington
31	Packers	Wisconsin

Next steps: Generate code with df_states | New interactive sheet

```
for i in range(0,32):
    df = df.replace(str(df_states['Team'][i]), str(df_states['State'][i]), regex=True)
df
```

	Cowboys	Giants	Eagles	Commanders	Bears	Lions	Packers	Vikings	Falcons	Pa
0	New Jersey	Texas	Texas	Texas	Michigan	Illinois	Illinois	Illinois	North Carolina	
1	Pennsylvania	Pennsylvania	New Jersey	New Jersey	Wisconsin	Wisconsin	Michigan	Michigan	Louisiana	Lo
2	Maryland	Maryland	Maryland	Pennsylvania	Minnesota	Minnesota	Minnesota	Wisconsin	Florida	
3	Texas	Texas	Arizona	Arizona	New York	Massachusetts	Georgia	Massachusetts	Ohio	
4	Indiana	Indiana	Florida	Florida	Florida	Louisiana	New York	Louisiana	Wisconsin	Wi
5	California	California	California	California	Georgia	New Jersey	North Carolina	New Jersey	Minnesota	Mi
6	Washington	Washington	Tennessee	Tennessee	North Carolina	Florida	Florida	Florida	Pennsylvania	Penns
7	Wisconsin	Michigan	Florida	Ohio	Washington	California	Arizona	California	Maryland	Penns
8			Maryland	Louisiana						

9 rows × 32 columns

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import re

headers = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4

url = "https://en.wikipedia.org/wiki/State_income_tax"
response = requests.get(url, headers=headers)
soup = BeautifulSoup(response.text, "html.parser")

# Find the correct table by caption text
tables = soup.find_all("table", {"class": "wikitable"})
target_table = None

for table in tables:
    caption = table.find("caption")
    if caption and "State individual income tax rates & brackets (2025)" in caption.text:
        target_table = table
        break

# Parse table rows
data = []
current_state = None

for row in target_table.find_all("tr")[1:]:
    cols = row.find_all(["td", "th"])
    cols = [c.get_text(strip=True) for c in cols]

    if not cols:
        continue

    # If first column has state name, update current_state
    if cols[0] and not cols[0].startswith("—"):
        current_state = cols[0]

    # Extract single filer rate (usually column 1)
    if len(cols) > 1:
        rate_text = cols[1]

        # Extract numeric percentage
        match = re.search(r"([\d.]+)%", rate_text)
        if match:
            rate = float(match.group(1))
            data.append((current_state, rate))
        elif "none" in rate_text.lower():
```

```
            data.append((current_state, 0.0))

# Convert to DataFrame
df_2 = pd.DataFrame(data, columns=["State", "Rate"])

# Get max rate per state
max_rates = df_2.groupby("State")["Rate"].max().reset_index()

for i in range(0,50):
    df = df.replace(str(max_rates['State'][i]), float(max_rates['Rate'][i]), regex=True).infer_objects(copy=False)

df = df.replace(str(''), float(0), regex=True).infer_objects(copy=False)

df
```

```
/tmp/ipykernel_6085/2527913917.py:56: FutureWarning: Downcasting behavior in `replace` is deprecated and will be remove
  df = df.replace(str(max_rates['State'][i]), float(max_rates['Rate'][i]), regex=True).infer_objects(copy=False)
/tmp/ipykernel_6085/2527913917.py:58: FutureWarning: Downcasting behavior in `replace` is deprecated and will be remove
  df = df.replace(str(''), float(0), regex=True).infer_objects(copy=False)
```

	Cowboys	Giants	Eagles	Commanders	Bears	Lions	Packers	Vikings	Falcons	Panthers	...	Jaguars	Titans	Chiefs
0	10.75	0.00	0.00	0.00	4.25	4.95	4.95	4.95	4.25	5.39	...	0.00	0.00	0.00
1	3.07	3.07	10.75	10.75	7.65	7.65	4.25	4.25	3.00	3.00	...	3.00	3.00	13.30
2	5.75	5.75	5.75	3.07	9.85	9.85	9.85	7.65	0.00	0.00	...	0.00	0.00	4.40
3	0.00	0.00	2.50	2.50	10.90	9.00	5.39	9.00	2.75	2.75	...	5.75	5.75	2.50
4	3.00	3.00	0.00	0.00	0.00	3.00	10.90	3.00	7.65	7.65	...	2.75	2.75	4.25
5	13.30	13.30	13.30	13.30	5.39	10.75	4.25	10.75	9.85	9.85	...	4.40	9.00	9.00
6	0.00	0.00	0.00	0.00	4.25	0.00	0.00	0.00	3.07	3.07	...	0.00	0.00	10.75
7	7.65	4.25	0.00	2.75	0.00	13.30	2.50	13.30	5.75	3.07	...	10.75	10.75	3.07
8	0.00	0.00	5.75	3.00	0.00	0.00	0.00	0.00	0.00	0.00	...	0.00	0.00	13.30

9 rows × 32 columns

```
effective_tax = df.sum(axis=0, numeric_only= True)
effective_tax = pd.DataFrame(effective_tax/1800)
effective_tax.columns = ['Tax Rate']
effective_tax
```

	Tax Rate
Cowboys	0.024178
Giants	0.016317
Eagles	0.021139
Commanders	0.019650
Bears	0.023494
Lions	0.032500
Packers	0.023383
Vikings	0.029389
Falcons	0.020178
Panthers	0.019322
Saints	0.021161
Buccaneers	0.016856
Cardinals	0.022122
Rams	0.026417
49ers	0.027744
Seahawks	0.025872
Ravens	0.013311
Browns	0.017994
Steelers	0.019861
Bengals	0.014800
Texans	0.017883
Colts	0.019772
Jaguars	0.014806
Titans	0.017361
Chiefs	0.033650
Raiders	0.029083
Chargers	0.026472
Broncos	0.026639
Bills	0.026083
Dolphins	0.036583
Patriots	0.028667
Jets	0.027556

Next steps: Generate code with `effective_tax` New interactive sheet

Merging Dataframes

```
#transforming schedule into home team and respective away games columns
df_schedule_long = df_opponents.melt(var_name='Home Team', value_name='Away Team')

# replacing empty strings with NAs
df_schedule_long['Away Team'] = df_schedule_long['Away Team'].replace('', pd.NA)

#droppping missing away team rows
df_schedule_long = df_schedule_long.dropna(subset=['Away Team'])
```

```
#checking columns
display(df_schedule_long.head(16))
```

	Home Team	Away Team
0	Cowboys	Giants
1	Cowboys	Eagles
2	Cowboys	Commanders
3	Cowboys	Texans
4	Cowboys	Colts
5	Cowboys	Rams
6	Cowboys	Seahawks
7	Cowboys	Packers
9	Giants	Cowboys
10	Giants	Eagles
11	Giants	Commanders
12	Giants	Texans
13	Giants	Colts
14	Giants	Rams
15	Giants	Seahawks
16	Giants	Lions

```
# getting count of home and away games for each team
home_games_count = df_schedule_long['Home Team'].value_counts().reset_index()
home_games_count.columns = ['Team', 'Home Games']
# standardizing home team names
home_games_count['Team'] = home_games_count['Team'].apply(lambda x: x.split()[-1])

away_games_count = df_schedule_long['Away Team'].value_counts().reset_index()
away_games_count.columns = ['Team', 'Away Games']
# standardizing away team names
away_games_count['Team'] = away_games_count['Team'].apply(lambda x: x.split()[-1])

#merging home and away game counts
df_games_count = pd.merge(home_games_count, away_games_count, on='Team', how='outer').fillna(0)

#changing game counts to integers
df_games_count['Home Games'] = df_games_count['Home Games'].astype(int)
df_games_count['Away Games'] = df_games_count['Away Games'].astype(int)

df_games_count
```

	Team	Home Games	Away Games
0	49ers	8	8
1	Bears	8	8
2	Bengals	9	8
3	Bills	9	7
4	Broncos	8	7
5	Browns	8	8
6	Buccaneers	8	10
7	Cardinals	8	9
8	Chargers	8	9
9	Chiefs	9	9
10	Colts	9	7
11	Commanders	9	10
12	Cowboys	8	8
13	Dolphins	9	8
14	Eagles	9	10
15	Falcons	8	7
16	Giants	8	9
17	Jaguars	8	9
18	Jets	9	8
19	Lions	8	9
20	Packers	8	8
21	Panthers	8	8
22	Patriots	8	10
23	Raiders	9	6
24	Rams	8	9
25	Ravens	9	10
26	Saints	8	9
27	Seahawks	8	9
28	Steelers	9	8
29	Texans	9	7
30	Titans	8	9
31	Vikings	8	7

Next steps: Generate code with df_games_count | New interactive sheet

```
#merging players with states
df_merged = pd.merge(df_top_1000, df_states, on='Team', how='left')
#merging with main table
merged_teams = pd.merge(df_merged, df_games_count, on='Team', how='left').fillna(0)
merged_teams
```

	Player	Pos.	Team	Total Value	APY	TotalGuaranteed	Avg.Guarantee/Year	% Guaranteed	State	Hom Game
0	Dak Prescott	QB	Cowboys	240000000.0	60000000.0	129000000.0	32250000.0	53.8	Texas	
1	Trevor Lawrence	QB	Jaguars	275000000.0	55000000.0	142000000.0	28400000.0	51.6	Florida	
2	Joe Burrow	QB	Bengals	275000000.0	55000000.0	146510000.0	29302000.0	53.3	Ohio	
3	Josh Allen	QB	Bills	330000000.0	55000000.0	147000000.0	24500000.0	44.5	New York	
4	Jordan Love	QB	Packers	220000000.0	55000000.0	100800000.0	25200000.0	45.8	Wisconsin	
...	...	...	...	...	...	...	...	...	...	
995	Kevin Winston Jr.	S	Titans	6404992.0	1601248.0	1298176.0	324544.0	20.3	Tennessee	
996	Matt Gay	K	Raiders	1600000.0	1600000.0	1350000.0	1350000.0	84.4	Nevada	
997	Renardo Green	CB	49ers	6389874.0	1597469.0	3347629.0	836907.0	52.4	California	
998	Emanuel Wilson	RB	Seahawks	1595000.0	1595000.0	550000.0	550000.0	34.5	Washington	
999	John Michael Schmitz	C	Giants	6373334.0	1593334.0	3424849.0	856212.0	53.7	New Jersey	

1000 rows × 11 columns

Next steps: Generate code with `merged_teams` New interactive sheet

```
# Aggregating all of this into one main table
aggregated_data = []
for name, group in merged_teams.groupby(['Team', 'State', 'Home Games', 'Away Games']):
    team, state, home_games, away_games = name
    players_info = group[['Player', 'Pos.', 'Total Value', 'APY', 'TotalGuaranteed', 'Avg.Guarantee/Year', '% Guarante
    aggregated_data.append({
        'Team': team,
        'State': state,
        'Home_Games': home_games,
        'Away_Games': away_games,
        'Players': players_info
    })
team_player_schedule = pd.DataFrame(aggregated_data)
team_player_schedule
```

	Team	State	Home_Games	Away_Games	Players
0	49ers	California	8	8	[{'Player': 'Brock Purdy', 'Pos.': 'QB', 'Tota...
1	Bears	Illinois	8	8	[{'Player': 'Montez Sweat', 'Pos.': 'EDGE', 'T...
2	Bengals	Ohio	9	8	[{'Player': 'Joe Burrow', 'Pos.': 'QB', 'Total...
3	Bills	New York	9	7	[{'Player': 'Josh Allen', 'Pos.': 'QB', 'Total...
4	Broncos	Colorado	8	7	[{'Player': 'Jaylen Waddle', 'Pos.': 'WR', 'To...
5	Browns	Ohio	8	8	[{'Player': 'Deshaun Watson', 'Pos.': 'QB', 'T...
6	Buccaneers	Florida	8	10	[{'Player': 'Baker Mayfield', 'Pos.': 'QB', 'T...
7	Cardinals	Arizona	8	9	[{'Player': 'Josh Sweat', 'Pos.': 'EDGE', 'Tot...
8	Chargers	California	8	9	[{'Player': 'Justin Herbert', 'Pos.': 'QB', 'T...
9	Chiefs	Missouri	9	9	[{'Player': 'Patrick Mahomes', 'Pos.': 'QB', '...
10	Colts	Indiana	9	7	[{'Player': 'Daniel Jones', 'Pos.': 'QB', 'Tot...
11	Commanders	Maryland	9	10	[{'Player': 'Laremy Tunsil', 'Pos.': 'LT', 'To...
12	Cowboys	Texas	8	8	[{'Player': 'Dak Prescott', 'Pos.': 'QB', 'Tot...
13	Dolphins	Florida	9	8	[{'Player': 'Malik Willis', 'Pos.': 'QB', 'Tot...
14	Eagles	Pennsylvania	9	10	[{'Player': 'Jalen Hurts', 'Pos.': 'QB', 'Tota...
15	Falcons	Georgia	8	7	[{'Player': 'Jake Matthews', 'Pos.': 'LT', 'To...
16	Giants	New Jersey	8	9	[{'Player': 'Brian Burns', 'Pos.': 'EDGE', 'To...
17	Jaguars	Florida	8	9	[{'Player': 'Trevor Lawrence', 'Pos.': 'QB', '...
18	Jets	New Jersey	9	8	[{'Player': 'Garrett Wilson', 'Pos.': 'WR', 'T...
19	Lions	Michigan	8	9	[{'Player': 'Jared Goff', 'Pos.': 'QB', 'Total...
20	Packers	Wisconsin	8	8	[{'Player': 'Jordan Love', 'Pos.': 'QB', 'Tota...
21	Panthers	North Carolina	8	8	[{'Player': 'Jaelan Phillips', 'Pos.': 'EDGE',...
22	Patriots	Massachusetts	8	10	[{'Player': 'Milton Williams', 'Pos.': 'IDL', ...
23	Raiders	Nevada	9	6	[{'Player': 'Maxx Crosby', 'Pos.': 'EDGE', 'To...
24	Rams	California	8	9	[{'Player': 'Matt Stafford', 'Pos.': 'QB', 'To...
25	Ravens	Maryland	9	10	[{'Player': 'Lamar Jackson', 'Pos.': 'QB', 'To...
26	Saints	Louisiana	8	9	[{'Player': 'Chase Young', 'Pos.': 'EDGE', 'To...
27	Seahawks	Washington	8	9	[{'Player': 'Jaxon Smith-Njigba', 'Pos.': 'WR'...
28	Steelers	Pennsylvania	9	8	[{'Player': 'T.J. Watt', 'Pos.': 'EDGE', 'Tota...
29	Texans	Texas	9	7	[{'Player': 'Will Anderson Jr.', 'Pos.': 'EDGE...
30	Titans	Tennessee	8	9	[{'Player': 'Jeffery Simmons', 'Pos.': 'IDL', ...
31	Vikings	Minnesota	8	7	[{'Player': 'Justin Jefferson', 'Pos.': 'WR', ...

Next steps: Generate code with `team_player_schedule` New interactive sheet

```
# Ensurig mascot name is extracted from full team name
def standardize_nfl_team_name(full_name):
    # if name is already a mascot
    if ' ' not in full_name:
        return full_name
    return full_name.split()[-1]

# List of full team names from df_opponents columns
teams_full_names = df_opponents.columns.tolist()

# creating dictionary
team_opponents_dict = {}
for full_name in teams_full_names:
```

```
        # list of opponents for each full team name
        opponents_list = df_opponents[full_name].tolist()
        cleaned_opponents = [opp.strip() for opp in opponents_list if pd.notna(opp) and opp.strip()]
        team_opponents_dict[full_name] = cleaned_opponents

    # Mapping full team names to their mascot names
    full_team_name_to_mascot_map = {full_name: standardize_nfl_team_name(full_name) for full_name in teams_full_names}
```

```
    # mapping mascot team names to their states
    team_mascot_to_state_map = df_states.set_index('Team')['State'].to_dict()

    # creating a dictionary
    team_away_game_states = {}

    # iterating through team_opponents_dict
    for home_team_full_name, away_opponents_from_schedule in team_opponents_dict.items():
        # getting mascot name for home team
        home_team_mascot = full_team_name_to_mascot_map.get(home_team_full_name)
        if home_team_mascot:
            unique_away_states = set() # Storing unique states
            for opponent_name_from_schedule in away_opponents_from_schedule:
                if opponent_name_from_schedule:
                    # Standardize the opponent name for consistency
                    opponent_mascot_standardized = standardize_nfl_team_name(opponent_name_from_schedule)
                    # Look up state for away team using mascot name
                    if opponent_mascot_standardized and opponent_mascot_standardized in team_mascot_to_state_map:
                        unique_away_states.add(team_mascot_to_state_map[opponent_mascot_standardized])

            team_away_game_states[home_team_mascot] = sorted(list(unique_away_states)) # Storing sorted list

    # Converting dictionary to df for future merging
    df_away_game_states = pd.DataFrame(list(team_away_game_states.items()), columns=['Team', 'Away_Game_States'])

    df_away_game_states
```

	Team	Away_Game_States
0	Cowboys	[California, Indiana, Maryland, New Jersey, Pe...
1	Giants	[California, Indiana, Maryland, Michigan, Penn...
2	Eagles	[Arizona, California, Florida, Maryland, New J...
3	Commanders	[Arizona, California, Florida, Louisiana, New ...
4	Bears	[Florida, Georgia, Michigan, Minnesota, New Yo...
5	Lions	[California, Florida, Illinois, Louisiana, Mas...
6	Packers	[Arizona, Florida, Georgia, Illinois, Michigan...
7	Vikings	[California, Florida, Illinois, Louisiana, Mas...
8	Falcons	[Florida, Louisiana, Maryland, Minnesota, Nort...
9	Panthers	[Florida, Georgia, Louisiana, Minnesota, Ohio,...
10	Saints	[Florida, Georgia, Illinois, Maryland, Michiga...
11	Buccaneers	[Georgia, Illinois, Louisiana, Maryland, Michi...
12	Cardinals	[California, Colorado, Florida, Maryland, Neva...
13	Rams	[Arizona, California, Louisiana, Missouri, New...
14	49ers	[Arizona, California, Georgia, Missouri, New J...
15	Seahawks	[Arizona, California, Colorado, Maryland, Neva...
16	Ravens	[Florida, Georgia, Maryland, North Carolina, O...
17	Browns	[Indiana, Maryland, Massachusetts, Ohio, Penns...
18	Steelers	[Florida, Louisiana, Maryland, New Jersey, Ohi...
19	Bengals	[Florida, Louisiana, Maryland, Massachusetts, ...
20	Texans	[Florida, Indiana, Maryland, Minnesota, Missou...
21	Colts	[California, Florida, Maryland, Ohio, Pennsylv...
22	Jaguars	[Colorado, Indiana, Maryland, New Jersey, Ohio...
23	Titans	[Florida, Indiana, Maryland, Massachusetts, Ne...
24	Chiefs	[Arizona, California, Colorado, Massachusetts,...
25	Raiders	[California, Colorado, Florida, Maryland, Miss...
26	Chargers	[Arizona, California, Colorado, Indiana, Massa...
27	Broncos	[California, Florida, Maryland, Missouri, Neva...
28	Bills	[California, Florida, Illinois, Massachusetts,...
29	Dolphins	[California, Colorado, Massachusetts, Minnesot...
30	Patriots	[California, Florida, Illinois, Michigan, Miss...
31	Jets	[Arizona, California, Florida, Illinois, Massa...

Next steps: Generate code with df_away_game_states | New interactive sheet

```
#performing final merge to create finished dataframe
final_df = pd.merge(team_player_schedule, df_away_game_states, on='Team', how='left')

#removing original columns for players to include row for each individual player
final_df = final_df.explode('Players')
final_df = pd.concat([
    final_df.drop(['Players'], axis=1),
    final_df['Players'].apply(pd.Series)
], axis=1)

final_df.to_csv('final_dataframe.csv')
final_df
```

	Team	State	Home_Games	Away_Games	Away_Game_States	Player	Pos.	Total Value	APY	TotalGuarante
0	49ers	California	8	8	[Arizona, California, Georgia, Missouri, New J...	Brock Purdy	QB	265000000.0	53000000.0	100000000
0	49ers	California	8	8	[Arizona, California, Georgia, Missouri, New J...	Nick Bosa	EDGE	170000000.0	34000000.0	88000000
0	49ers	California	8	8	[Arizona, California, Georgia, Missouri, New J...	Brandon Aiyuk	WR	120000000.0	30000000.0	45000000
0	49ers	California	8	8	[Arizona, California, Georgia, Missouri, New J...	Trent Williams	LT	82660000.0	27553333.0	26900000
0	49ers	California	8	8	[Arizona, California, Georgia, Missouri, New J...	Fred Warner	LB	63000000.0	21000000.0	18150000
...	...	...	...	...	...	...	...	...	...	
31	Vikings	Minnesota	8	7	[California, Florida, Illinois, Louisiana, Mas...	Tavierre Thomas	CB	4600000.0	2300000.0	1950000
31	Vikings	Minnesota	8	7	[California, Florida, Illinois, Louisiana, Mas...	Domonique Orange	IDL	7020028.0	1755007.0	0
31	Vikings	Minnesota	8	7	[California, Florida, Illinois, Louisiana, Mas...	Andrew DePaola	LS	1725000.0	1725000.0	1725000
31	Vikings	Minnesota	8	7	[California, Florida, Illinois, Louisiana, Mas...	Caleb Tiernan	T	6726012.0	1681503.0	0
31	Vikings	Minnesota	8	7	[California, Florida, Illinois, Louisiana, Mas...	Jakobe Thomas	S	6726012.0	1681503.0	0

1000 rows × 12 columns

Next steps: Generate code with `final_df` New interactive sheet

```
#merging final dataframe with effective tax rates

# Making team names a column for merging
effective_tax_processed = effective_tax.reset_index()
effective_tax_processed.rename(columns={'index': 'Full_Team_Name'}, inplace=True)

# standardizing to mascot name to match
effective_tax_processed['Team'] = effective_tax_processed['Full_Team_Name'].apply(standardize_nfl_team_name)

# final merge
final_df = pd.merge(final_df, effective_tax_processed[['Team', 'Tax Rate']], on='Team', how='left')

display(final_df.head())
```

	Team	State	Home_Games	Away_Games	Away_Game_States	Player	Pos.	Total Value	APY	TotalGuaranteed	Av
0	49ers	California	8	8	[Arizona, California, Georgia, Missouri, New J...	Brock Purdy	QB	265000000.0	53000000.0	100000000.0	
1	49ers	California	8	8	[Arizona, California, Georgia, Missouri, New J...	Nick Bosa	EDGE	170000000.0	34000000.0	88000000.0	

```
# federal tax brackets for single filers (2025)
def federal_tax_rate(income):
    brackets = [(11925,0.10),(48475,0.12),(103350,0.22),(197300,0.24),(250525,0.32),(626350,0.35),(float('inf'),0.37)]
    tax, prev = 0, 0
    for limit, rate in brackets:
        if income <= prev: break
        tax += (min(income, limit) - prev) * rate
        prev = limit
    return tax / income if income > 0 else 0

# state tax rate lookup from earlier scrape
# state tax rate lookup from earlier scrape
state_tax_lookup = {state: rate/100 for state, rate in zip(max_rates['State'], max_rates['Rate'])}
# jock tax: 2.5% of salary per away state
def jock_tax_rate(away_states):
    if not isinstance(away_states, list): return 0
    return sum(0.025 * state_tax_lookup.get(s, 0) for s in away_states)

final_df['Federal Tax Rate']   = final_df['APY'].apply(federal_tax_rate)
final_df['Home State Tax Rate']= final_df['Tax Rate']
final_df['Jock Tax Rate']      = final_df['Away_Game_States'].apply(jock_tax_rate)
final_df['Effective Tax Rate'] = final_df['Federal Tax Rate'] + final_df['Home State Tax Rate'] + final_df['Jock Tax R
final_df['Total Tax ($)']      = final_df['Effective Tax Rate'] * final_df['APY']

final_df.to_csv('final_datatframe_w_taxes.csv')
display(final_df[['Player','Team','State','APY','Federal Tax Rate','Home State Tax Rate','Jock Tax Rate','Effective Ta
```

	Player	Team	State	APY	Federal Tax Rate	Home State Tax Rate	Jock Tax Rate	Effective Tax Rate
0	Brock Purdy	49ers	California	53000000.0	0.369189	0.027744	0.00916	0.406094
1	Nick Bosa	49ers	California	34000000.0	0.368736	0.027744	0.00916	0.405640
2	Brandon Aiyuk	49ers	California	30000000.0	0.368567	0.027744	0.00916	0.405472
3	Trent Williams	49ers	California	27553333.0	0.368440	0.027744	0.00916	0.405345
4	Fred Warner	49ers	California	21000000.0	0.367953	0.027744	0.00916	0.404858
5	Osa Odighizuwa	49ers	California	20000000.0	0.367851	0.027744	0.00916	0.404755
6	George Kittle	49ers	California	19100000.0	0.367750	0.027744	0.00916	0.404654